

MultiThreading

Part-1: Introduction to MultiThreading (2 days)

Q. What is multithreading?

Answer: Multithreading is a programming technique in Java that allows multiple threads to run concurrently (at the same time) within a single program. Each thread is a lightweight subprocess, the smallest unit of processing.

Q. Why do we use multithreading? Give real time examples.

Answer: We use multithreading to perform multiple tasks simultaneously, improve performance, ensure responsiveness, and make better use of CPU resources. It helps in writing efficient and fast applications, especially when there are tasks that can run independently.

Reason	Explanation
1. Better CPU Utilization	Modern CPUs have multiple cores. Threads can run in parallel, improving performance.
2. Improved Performance	Tasks like file download, image processing, or network calls can run in the background without blocking the main program.
3. Faster Execution	Multiple threads reduce overall execution time when tasks can be done independently.
4. Responsiveness	In UI-based applications, long-running operations can be run in the background to avoid freezing the interface.
5. Resource Sharing	Threads share the same memory space, which makes communication between them easier than separate processes.

Real time examples of multi threading

Real-Time Scenario	How Multithreading Helps
1. Web Server	A web server like Apache handles multiple client requests simultaneously using threads — each request runs in a separate thread.
2. Video Games	Different threads for rendering graphics, playing sound, managing input, and physics simulation — all running at the same time.
3. Online Payment App	One thread verifies card details, another connects to the bank, another shows a progress bar — all in parallel.
4. Media Player	One thread plays video, another plays audio, another responds to user actions (like pause/seek).
5. Chat Application	One thread listens for new messages, another sends messages, another updates the chat UI.
6. File Download Manager	Each file download runs in a separate thread, allowing multiple files to be downloaded at once.
7. Compiler/IDE	Compiles code in the background while you continue typing, using background threads.
8. Background Tasks in Mobile Apps	Fetching data from the server, syncing contacts, or notifications — all done using background threads to avoid freezing the UI.
9. ATM Machine Software	One thread reads card, another checks balance, another prints receipt — handled in parallel.

Real-Time Scenario

How Multithreading Helps

10. E-commerce Website

One thread checks stock, another updates inventory, another generates invoice, another sends email confirmation.

Q. What is a thread?

Answer: A **thread** is the **smallest unit of execution** in a Java program. It represents a **separate path of execution**, allowing you to run **multiple operations concurrently** within a single program.

Think of a thread as a **single worker** in a large factory, multiple workers (threads) can work on different tasks at the same time, sharing the same space (memory/resources).

We can say A **thread** is a lightweight subprocess that runs concurrently with other threads in the **same process**.

Q. What is Thread?

Answer: In Java, `java.lang.Thread` is a **class** provided by the **Java standard library** that allows you to **create and manage threads** in your program. Since it's in `java.lang`, it is **imported by default** in every Java program — no need for an explicit import statement.

We can say, `java.lang.Thread` is a **built-in Java class** used to define and control threads. It provides methods to start, run, and manage concurrent execution in a program.

Q. What is a single threaded application model?

Answer: A single-threaded application model is a type of program where only one thread (typically the main thread) is responsible for executing all tasks, one at a time, in a sequential order. This is common in **simple console applications**, especially those that do not perform tasks in parallel.
e.g:

```
import java.util.Scanner;

public class SingleThreadedBankApp {
    private static double balance = 0;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        while (true) {
            System.out.println("\n=== Banking System ===");
            System.out.println("1. Deposit");
            System.out.println("2. Withdraw");
            System.out.println("3. Check Balance");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");

            int choice = sc.nextInt();

            switch (choice) {
                case 1:
                    System.out.print("Enter amount to deposit: ");
                    double deposit = sc.nextDouble();
                    balance += deposit;
                    System.out.println("Deposited: ₹" + deposit);
```

```

        break;

    case 2:
        System.out.print("Enter amount to withdraw: ");
        double withdraw = sc.nextDouble();
        if (withdraw <= balance) {
            balance -= withdraw;
            System.out.println("Withdrawn: ₹" + withdraw);
        } else {
            System.out.println("Insufficient balance.");
        }
        break;

    case 3:
        System.out.println("Current Balance: ₹" + balance);
        break;

    case 4:
        System.out.println("Exiting... Thank you!");
        System.exit(0);

    default:
        System.out.println("Invalid choice. Try again.");
    }
}
}
}
}

```

This application simulates a basic banking operation using only the `main` thread.

Q. What is a multi threaded application model?

Answer: A **multithreaded application model** is one in which an application uses **multiple threads** to perform tasks **concurrently**. This model improves performance and responsiveness, especially in **real-time or resource-intensive** applications.

```

class OrderTaking extends Thread {
    public void run() {
        System.out.println("OrderTaking: Receiving orders...");
        try {
            Thread.sleep(2000); // simulate time taken to take orders
        } catch (InterruptedException e) {
            System.out.println("OrderTaking Interrupted");
        }
        System.out.println("OrderTaking: Orders received.");
    }
}

class PaymentProcessing extends Thread {
    public void run() {
        System.out.println("PaymentProcessing: Processing payments...");
    }
}

```

```

    try {
        Thread.sleep(3000); // simulate payment time
    } catch (InterruptedException e) {
        System.out.println("PaymentProcessing Interrupted");
    }
    System.out.println("PaymentProcessing: Payments completed.");
}
}

class DeliveryTracking extends Thread {
    public void run() {
        System.out.println("DeliveryTracking: Updating delivery status...");
        try {
            Thread.sleep(4000); // simulate time taken to track
        } catch (InterruptedException e) {
            System.out.println("DeliveryTracking Interrupted");
        }
        System.out.println("DeliveryTracking: Order delivered.");
    }
}

public class OnlineFoodDeliverySystem {
    public static void main(String[] args) {
        OrderTaking orderThread = new OrderTaking();
        PaymentProcessing paymentThread = new PaymentProcessing();
        DeliveryTracking deliveryThread = new DeliveryTracking();

        orderThread.start();
        paymentThread.start();
        deliveryThread.start();
    }
}

```

Application

Web browsers

Online games

Banking systems

Web servers

Media players

Use of Multithreading

One thread for rendering, another for downloading

Threads for player input, rendering, networking

Separate threads for transaction, balance update, fraud detection

Handling multiple client requests in parallel

Play music while loading next song, updating UI, etc.

Q. What is the difference between single threaded application and multi-threaded application?

Answer:

Single-Threaded Application

Uses only one thread (usually main) to execute the entire program sequentially.

Tasks run **one after another**.

Multi-Threaded Application

Uses multiple threads to execute different parts of the program concurrently.

Tasks can run **in parallel**.

Single-Threaded Application

Slower for complex or long-running tasks.

UI or main process **freezes** if a task takes long.

Utilizes **a single CPU core** mostly.

Simple to implement and debug.

Simple console apps, calculators, scripts.

Multi-Threaded Application

Faster and more responsive for I/O or CPU-intensive tasks.

UI remains **responsive** because background tasks can run in parallel.

Can **utilize multiple CPU cores** efficiently.

More complex to implement (requires synchronization, thread safety).

Web servers, games, media players, real-time apps.

An application which uses only a single thread for executing all its task then its a single thread model application. In Single Threaded Application the execution is sequential flow of execution. Here if the first task is completed successfully the only second task execution is started.

Multi-Threaded Model Application: An application which uses multiple threads for executing all its task then its a multi-thread model application. Here we have a minimum of two threads. First is main thread and the another is custom thread. In Multi-Threaded Application, the execution is concurrent flow of execution.

Q. Which application is better, single threaded application or multi-threaded application and Why?

Answer: Multi-threaded applications are generally better in modern software development because they support **parallelism**, **scalability**, and **responsiveness**.

Q. Explain the life cycle of thread?

Answer: In Java, a thread goes through **five main stages** in its **life cycle**, from creation to termination. These stages are managed by the **JVM** and the **thread scheduler**.

- **New**
- **Runnable**
- **Running**
- **Blocked / Waiting / Timed Waiting**
- **Terminated**

1. NEW State

A thread is in the NEW state when it is created but not started.

e.g: A delivery task is created but not started yet.

```
class DeliveryTask extends Thread {
    public void run() {
        System.out.println("Delivering package...");
    }
}

public class NewStateExample {
    public static void main(String[] args) {
        DeliveryTask t = new DeliveryTask(); // NEW
        System.out.println("Thread State: " + t.getState()); // NEW
    }
}
```

```
}
```

2. RUNNABLE State

After start() is called, the thread enters the RUNNABLE state. It's waiting for CPU to pick it up.

e.g: Delivery thread is ready to start but waiting for CPU time.

```
public class RunnableStateExample {  
    public static void main(String[] args) {  
        Thread t = new Thread(() -> {  
            System.out.println("Delivering item...");  
        });  
  
        t.start(); // RUNNABLE  
        System.out.println("Thread State (after start): " + t.getState()); // RUNNABLE or RUNNING  
    }  
}
```

Output:

Thread State (after start): RUNNABLE

Delivering item...

3. RUNNING State

- The thread scheduler picks the thread from the RUNNABLE pool, and it starts executing.
- You can't force RUNNING. The JVM decides when a RUNNABLE thread starts running.

e.g: Thread is delivering the package now.

```
public class RunningStateExample {  
    public static void main(String[] args) {  
        Thread t = new Thread(() -> {  
            System.out.println("Thread is running (delivering package)..."); // RUNNING  
        });  
  
        t.start();  
    }  
}
```

4. BLOCKED State

When a thread tries to access a synchronized resource already locked by another thread, it enters the BLOCKED state.

e.g: Two workers trying to access the same delivery van. One is blocked.

```
class Van {  
    synchronized void load() {  
        System.out.println(Thread.currentThread().getName() + " loading van...");  
        try { Thread.sleep(3000); } catch (InterruptedException e) {}  
        System.out.println(Thread.currentThread().getName() + " done loading.");  
    }  
}
```

```

public class BlockedStateExample {
    public static void main(String[] args) {
        Van van = new Van();

        Thread t1 = new Thread(() -> van.load(), "Worker1");//named the thread as Worker1
        Thread t2 = new Thread(() -> van.load(), "Worker2");//named the thread as worker2

        t1.start();
        try { Thread.sleep(100); } catch (Exception e) {}
        t2.start(); // here t2 i.e., worker2 is BLOCKED until t1 releases the lock
    }
}

```

Output:

```

Worker1 loading van...
Worker1 done loading.
Worker2 loading van...
Worker2 done loading.

```

5. WAITING State

A thread enters WAITING when it waits indefinitely for another thread to signal (notify()).

e.g: Waiter waits for chef to finish cooking.

```

class Kitchen {
    synchronized void waiterWaiting() {
        try {
            System.out.println("Waiter is waiting...");
            wait(); // WAITING
            System.out.println("Waiter received the signal.");
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    synchronized void chefNotify() {
        notify(); // Wakes up waiter
    }
}

```

```

public class WaitingStateExample {
    public static void main(String[] args) throws InterruptedException {
        Kitchen kitchen = new Kitchen();

        Thread waiter = new Thread(() -> kitchen.waiterWaiting());
        waiter.start();

        Thread.sleep(2000); // ensure waiter starts and waits

        Thread chef = new Thread(() -> kitchen.chefNotify());
        chef.start();
    }
}

```

```
}
```

6. TIMED_WAITING State

A thread enters TIMED_WAITING when it waits for a fixed amount of time using methods like `sleep()`, `join(timeout)`, or `wait(timeout)`.

e.g: Delivery person takes a break for 3 seconds.

```
public class TimedWaitingStateExample {
    public static void main(String[] args) {
        Thread t = new Thread(() -> {
            try {
                System.out.println("Taking a break...");
                Thread.sleep(3000); // TIMED_WAITING
                System.out.println("Break over.");
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });

        t.start();
    }
}
```

7. TERMINATED (Dead) State:

A thread is in the TERMINATED state after its `run()` method has completed or it is forcefully stopped.

e.g: The delivery task is complete.

```
public class TerminatedStateExample {
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> {
            System.out.println("Task done."); // RUNNING → TERMINATED
        });

        t.start();
        t.join(); // wait for completion

        System.out.println("Thread State: " + t.getState()); // TERMINATED
    }
}
```

State	Meaning	Triggered By
NEW	Thread created, not started	<code>new Thread()</code>
RUNNABLE	Thread started, waiting for CPU	<code>start()</code>
RUNNING	Thread picked by CPU, executing	Managed by JVM Scheduler
BLOCKED	Waiting to access a locked resource	synchronized block already locked

State	Meaning	Triggered By
WAITING	Waiting for another thread's signal (indefinite)	wait(), join()
TIMED_WAITING	Waiting for a specific time	sleep(), join(timeout), wait(time)
TERMINATED	Thread execution complete	run() finished or stop() called

Q. What are the names of different states of a thread?

Answer: In Java, the Thread class has an enum called Thread.State, which defines the names of the different thread states. These are the official thread states as per the Java Language Specification.

State Name	Description
NEW	The thread is created but not started yet.
RUNNABLE	The thread is ready to run but is waiting for CPU time.
BLOCKED	The thread is waiting to acquire a lock held by another thread.
WAITING	The thread is waiting indefinitely for another thread to perform a specific action.
TIMED_WAITING	The thread is waiting for a specific time before proceeding.
TERMINATED	The thread has finished execution (either normally or due to an exception).

```
public class ThreadStateExample {
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> {
            try {
                Thread.sleep(2000);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });

        System.out.println("State before start: " + t.getState()); // NEW
        t.start();
        System.out.println("State after start: " + t.getState()); // RUNNABLE or TIMED_WAITING
        t.join();
        System.out.println("State after termination: " + t.getState()); // TERMINATED
    }
}
```

Q. How do we create a thread?

Answer:

In Java, a thread can be created in two main ways:

1. By extending the Thread class:

You create a class that extends Thread, and override the run() method. Then you create an object of that class and call start() to begin the thread.

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread(); // Create thread object
        t1.start(); // Start the thread
    }
}
```

2. By implementing the Runnable interface:

You create a class that implements Runnable, then define the run() method, and finally pass that object to a Thread constructor.

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
        MyRunnable runnable = new MyRunnable();
        Thread t1 = new Thread(runnable); // Pass Runnable to Thread
        t1.start(); // Start the thread
    }
}
```

Using Lambda Expression (Java 8+)

If you're using Java 8 or above, you can use a lambda expression to create threads more concisely.

```
public class Main {
    public static void main(String[] args) {
        Thread t1 = new Thread(() -> {
            System.out.println("Thread using lambda is running...");
        });
        t1.start();
    }
}
```

Approach	Implements / Extends	Flexibility	Common Use Case
Extending Thread class	extends Thread	Less (can't extend another class)	Simple thread logic
Implementing Runnable	implements Runnable	More (can extend another class)	Preferred for reusability
Using Lambda (Java 8+)	Runnable via lambda	Concise	Simple thread tasks

Q. What is Runnable?

Answer: Runnable is a functional interface in Java that represents a task that can be executed by a thread. It is part of the java.lang package.

Structure:

```
@FunctionalInterface
public interface Runnable {
    public abstract void run();
}
```

It has only one method: run(), which you override to define the task the thread will perform.

Note:

- Java does not support multiple inheritance using classes. So if your class already extends another class, using Runnable is better.
- Runnable allows better separation of task (logic) and thread management.
- It is recommended when sharing resources between multiple threads.

Q. How do we create a thread using Runnable?

Answer:

Step-by-step:

1. Create a class that implements Runnable.
2. Override the run() method with your task logic.
3. Create a Thread object and pass your Runnable instance.
4. Call start() on the Thread.

```
class PrintTask implements Runnable {
    public void run() {
        System.out.println("Running a task using Runnable...");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Runnable task = new PrintTask(); // Step 1 & 2
        Thread thread = new Thread(task); // Step 3
        thread.start(); // Step 4
    }
}
```

Real-Time Example: Ticket Booking System

```
class TicketBooking implements Runnable {
    public void run() {
        System.out.println(Thread.currentThread().getName() + " is booking a ticket.");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Runnable booking = new TicketBooking();

        Thread user1 = new Thread(booking, "Alice");
```

```

Thread user2 = new Thread(booking, "Bob");

user1.start();
user2.start();
}
}

```

Note:

Since Java 8 – Runnable is a Functional Interface

You can use a lambda expression instead of writing a separate class:

```

public class Main {
    public static void main(String[] args) {
        Runnable r = () -> System.out.println("Lambda Runnable running...");
        Thread t = new Thread(r);
        t.start();
    }
}

```

Note:

- Runnable is used to define a task.
- It separates **task logic** from **thread mechanics**.
- Preferred over extending Thread for flexibility.
- Supports lambda expressions for concise code.
- Widely used in **real-time apps** like ticket booking, banking, etc.

Q. What is the difference between creating thread using Thread and creating thread using Runnable?

Answer:

Feature	Using Thread	Using Runnable
Inheritance	Must extend Thread , so can't extend any other class	Can still extend another class , since Java allows only single inheritance
Design	Tight coupling with Thread	Loose coupling, promotes better design (separation of task and execution)
Reusability	Less reusable – the class is bound to Thread	More reusable – the Runnable can be passed to different Threads
Best Practice	Not preferred for large applications	Preferred in real-world projects and for implementing thread pools
Flexibility	Cannot share the same instance easily among threads	Runnable instance can be shared among multiple threads for parallel processing

Scenario: Online Ticket Booking System

Requirement:

Multiple users are trying to book movie tickets at the same time. The booking system must ensure that tickets are not overbooked, and the logic for booking is shared across all users.

Using Thread (not ideal in this case):

Each user will be represented by a separate thread class, but since the logic is embedded inside the thread class itself, the logic can't be reused or shared efficiently.

```

class User1 extends Thread {
    int tickets = 1;

    public void run() {
        if (tickets > 0) {
            System.out.println(Thread.currentThread().getName() + " booked a ticket");
            tickets--;
        } else {
            System.out.println("No tickets left");
        }
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        User1 u1 = new User1();
        User1 u2 = new User1(); // new thread, separate instance

        u1.start();
        u2.start(); // This may still allow overbooking!
    }
}

```

Problem: Each thread has its own copy of tickets, so overbooking may happen.

Using Runnable (ideal for real-world apps):

Here, we create one shared booking system, and multiple user threads will attempt to access it.

```

class TicketBooking implements Runnable {
    int tickets = 1;

    public void run() {
        synchronized (this) { // thread-safe block
            if (tickets > 0) {
                System.out.println(Thread.currentThread().getName() + " booked a ticket");
                tickets--;
            } else {
                System.out.println(Thread.currentThread().getName() + ": No tickets left");
            }
        }
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        TicketBooking booking = new TicketBooking();

        Thread user1 = new Thread(booking, "User1");
        Thread user2 = new Thread(booking, "User2");

        user1.start();
        user2.start(); // Only one will succeed
    }
}

```

```
}  
}
```

Benefits of Runnable in this case:

Logic Reusability: TicketBooking can be used by multiple threads (users).

Thread Safety: Easy to synchronize shared logic.

Real-world Simulation: Mimics how ticket booking works in real applications (shared service accessed by many).

Q. Which way of thread creation is better and why?

Answer:

Using the **Runnable interface** is considered better than extending the Thread class in most real-world applications.

Reason	Explanation
Better Design (Separation of Concerns)	Runnable separates the task (what to do) from the thread (how to do it) . This leads to cleaner and modular code.
Allows Multiple Inheritance	Since Java supports only single inheritance , extending Thread means your class can't extend any other class. But if you use Runnable, your class can still extend another class.
Reusable Logic	A single Runnable instance can be shared by multiple threads, which is useful for handling shared resources (e.g., bank account, booking system).
Thread Pool Support	Executors and thread pools in modern Java concurrency APIs (like ExecutorService) require Runnable or Callable , not Thread.
Lightweight	Creating a thread using Runnable consumes less memory and gives more flexibility to manage threads.

Q. When does a thread starts executing?

Answer: A thread **starts executing** when you call the start() method on a Thread object.

e.g:

```
Thread t = new Thread();  
t.start(); // Thread enters the "Runnable" state and is eligible to run
```

Step	Description
new	The thread object is created, but the thread is not yet started.
start()	This call asks the JVM to schedule the thread for execution. The thread enters the Runnable state .
Runnable → Running	The thread is picked by the thread scheduler and starts executing the run() method.
After Execution	Once run() finishes, the thread enters the Terminated (Dead) state.

A thread starts executing only after calling the start() method, which internally calls the run() method on a new call stack.

Q. What is the relation between start() and run()?

Answer:

- The `start()` method is used to **create a new thread** and **automatically call the `run()` method** in that new thread.
- The `run()` method contains the **code that will be executed** by the thread.

Method	Role
<code>start()</code>	Prepares a new thread, tells JVM to schedule it, and internally calls <code>run()</code> in a new call stack/thread.
<code>run()</code>	The actual method that holds the task to be executed in the thread.

Note:

- If you call `run()` directly, no new thread is created — the code just runs in the current thread (usually `main`).
- To run `run()` in a new thread, you must call `start()`.

```
class MyTask extends Thread {
    public void run() {
        System.out.println("Running in thread: " + Thread.currentThread().getName());
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        MyTask t = new MyTask();

        t.run(); // Just calls run() in main thread
        t.start(); // Calls run() in a NEW thread
    }
}
```

Output: Running in thread: main // from `t.run()`
Running in thread: Thread-0 // from `t.start()`

Q. What are the methods available in Thread?

Answer:

The `Thread` class in Java provides many methods to control and interact with threads. These methods come from:

- `java.lang.Thread`
- Some inherited from `Object` and `Runnable`

Method	Description
<code>start()</code>	Starts a new thread and calls the <code>run()</code> method internally
<code>run()</code>	Contains the code to be executed in the thread
<code>sleep(ms)</code>	Pauses the thread for a given number of milliseconds
<code>join()</code>	Waits for another thread to finish execution
<code>isAlive()</code>	Returns <code>true</code> if the thread is still running
<code>getName()</code>	Returns the name of the thread
<code>setName(String name)</code>	Sets the thread's name
<code>getId()</code>	Returns the thread's unique ID
<code>getPriority()</code>	Returns thread priority (1 to 10)

Method	Description
setPriority(int priority)	Sets thread priority
yield()	Pauses the current thread and gives chance to others of the same priority
interrupt()	Interrupts a sleeping or waiting thread
isInterrupted()	Checks if the thread was interrupted
currentThread()	Static method: returns reference to the currently executing thread
activeCount()	Static method: returns number of active threads in current thread group

Deprecated Methods (Avoid using):

Method	Reason
stop()	Unsafe: can leave shared resources in inconsistent state
suspend() / resume()	Unsafe: may lead to deadlock

Q. What are the methods available in Runnable?

Answer: The Runnable interface in Java has only one method i.e. run().

Q. What is the use of start()?

Answer: The start() method is used to start a new thread of execution in Java.

It tells the JVM to create a new call stack and then calls the thread's run() method in that new thread.

When we call start() the following happens:

- The thread is moved from the New state to the Runnable state.
- The JVM's thread scheduler picks it up and allocates CPU time.
- The run() method is called internally in a separate thread.

NOTE:

Use start() to create a new thread and invoke run() asynchronously. Never call run() directly if your intention is to run code in a new thread.

Q. What is the use of run?

Answer:

- The run() method is part of the Runnable interface and is overridden when creating a thread.
- When a thread starts using start(), the JVM internally calls the run() method.
- The code inside run() is executed on a separate call stack (i.e., in a new thread) when started properly.

Q. Can we override run()?

Answer: Yes.

Q. Why do we override run?

Answer:

- The run() method is the entry point for code that runs in a new thread.
- When you create a thread (either by extending Thread or implementing Runnable), you override run() to tell the JVM what the thread should do when it is started using start().

Q. Can we override start()?

Answer: Yes.

Q. Why do we override start?

Answer:

Some developers override start() to add custom logging or setup, but that is not recommended. If you accidentally forget to call super.start(), your thread will never actually run in a new thread.

Q. Can we overload run()?

Answer:

- Java allows method overloading — so you can define multiple run() methods with different parameters.
- However, when a thread is started using start(), the JVM will only call the no-argument run() method, i.e., public void run().

Note:

Overloading run() is legal, but has no effect on thread behavior. Always override public void run() for multi-threaded logic.

Q. Can we overload start()?

Answer:

Yes, you can overload the start() method in Java.

Only the no-argument start() method from the Thread class is recognized by the JVM to actually start a new thread.